CSC902: Advanced Algorithms
Greedy and Dynamic Programming

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

Table of Contents

1. Greedy Algorithm

2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree

3. References

Greedy Algorithm

- A greedy algorithm obtains an **optimal solution** to an optimization problem by making a sequence of choices.
- At each decision point, the algorithm makes the choice that seems best at the moment; Thus, **does not always produce an optimal solution, but** sometimes it does.

Approach for greedy algorithm design:

1. Determine the **optimal substructure** of the problem.
2. Develop a recursive solution.
3. Show that **if greedy choice is made**, then **only one subproblem remains**.
4. Prove that it is always safe to make the greedy choice.
5. Develop a recursive algorithm that implements the greedy strategy.
6. Convert the recursive algorithm to an iterative algorithm.

Properties: greedy choice property and **optimal substructure property**

Greedy Algorithm: properties

- **greedy choice:** When considering which choice is to make, the **best looking** choice is made for the current problem, **without considering results from subproblems**.
- **optimal substructure:** if an **optimal solution** to the problem contains within it **optimal solutions to subproblems**; A greedy choice is **part of the final optimal solution**. We never discard an already selected choice during the iterations.

Huffman Encoding: Introduction(1/2)

- **Huffman Codes:** provide a method of encoding data efficiently. Used in data compression.
- If each character is coded using standard codes like ASCII or the Unicode, each character is rep- resented by a fixed-length (b) codeword of bits (e.g. 8 or 16 bits per character), then total number of bits required to encode a n character string is nb
- Fixed-length codewords are easy to decipher due to the same length of each character. However, fixed-length codes may not be the most efficient from the perspective of minimizing the total quantity of data.
- Example 1 -Fixed Length: Let alphabet $C = \{a, b, c, d\}$ and the characters a, b, c, d are represented by the 2-bit code $00, 01, 10, 11$ respectively. Consider a string made up of only the alplabets from C; a 10 length string *abacdaacac*. Using the 2-bit encoding, the string is represeted as 00010010110000100010, i.e, it takes $nb = 20$ bits for encoding.

Huffman Encoding: Introduction(2/2)

- Note that deciphering the encoded string is easy; since each character is 2 bit (fixed length), pick 2 bits at a time and decipher it as the corresponding charatcter.
- Example 2 -Variable length: If relative probabilities of occurrences of each charater is known beforehand (when known text to be compressed and send through a network channel), use smaller number of bits to encode frequently occurring characters and bigger number of bits for less frequently occurring alphabets in the string.
- **Example 2 -Variable length:** The encoded string is 01100101110010010 using the following encoding.

Character	a	b	c	d
Probability	0.60	0.05	0.30	0.05
Variable-Length Codeword	0	110	10	111

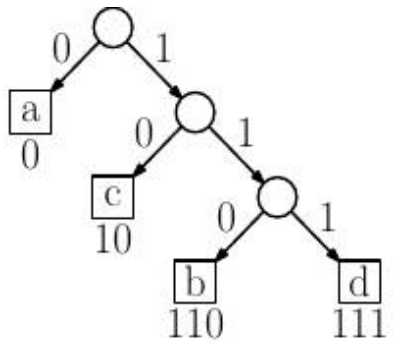
- In Example 2, the string length is 17, thus saving 3 bits from Example 1. Expected number of bits for variable length codeword for a string of n length is n times expected codelength, i.e, $n(0.60.1+0.05.3+0.30.2+0.05.3) = n(0.60+0.15+0.60+0.15) = 1.5n$

Huffman Encoding: Prefix code

Prefix code: Mapping of codewords to characters so that no codeword is a prefix of another.

- if two codewords (variable length) share a common prefix, e.g. $a \rightarrow 001$ and $b \rightarrow 00101$, then when we see $00101\cdots$ for an encoded string, how do we know whether the first character of the encoded message is a or b?
- In order to decipher an encoded string without ambiguity, need to generate prefix codes.
- **Representation of a binary prefix code:** a binary tree in which the **codewords** are the **leaves of the tree**, and where a left branch means 0 and a right branch means 1. The **length** of a codeword is just its **depth in the tree**.

Character	a	b	c	d
Codeword	0	110	10	111



Huffman Encoding Problem

Expected encoding length of string:

- Let $p(x)$ be the probability of seeing a character $x \in C$ (given), and let $d_T(x)$ denotes length of the codeword (depth in the tree) relative to some prefix tree T.
- The expected number of bits needed to encode a text with n characters is $B(T) = n \sum_{x \in C} p(x) d_T(x)$.

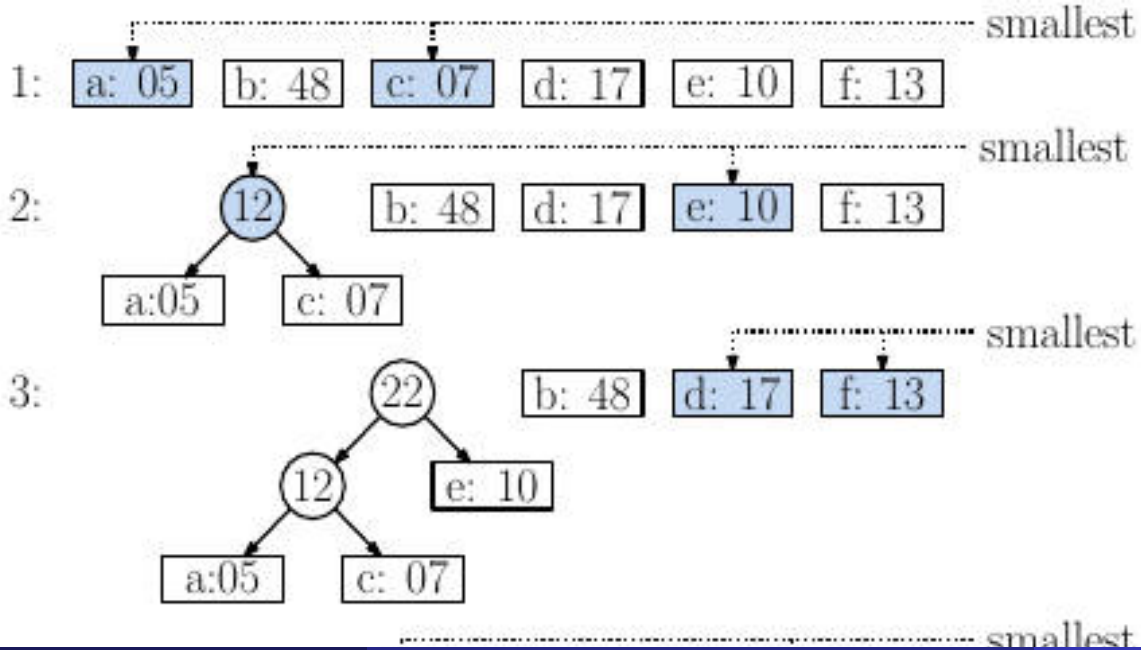
Optimal code generation Problem: Given an alphabet C and the probabilities $p(x)$ of occurrence for each character $x \in C$, compute a prefix code T that minimizes the expected length of the encoded bit-string, $B(T)$.

Solution: build an optimal Prefix tree T

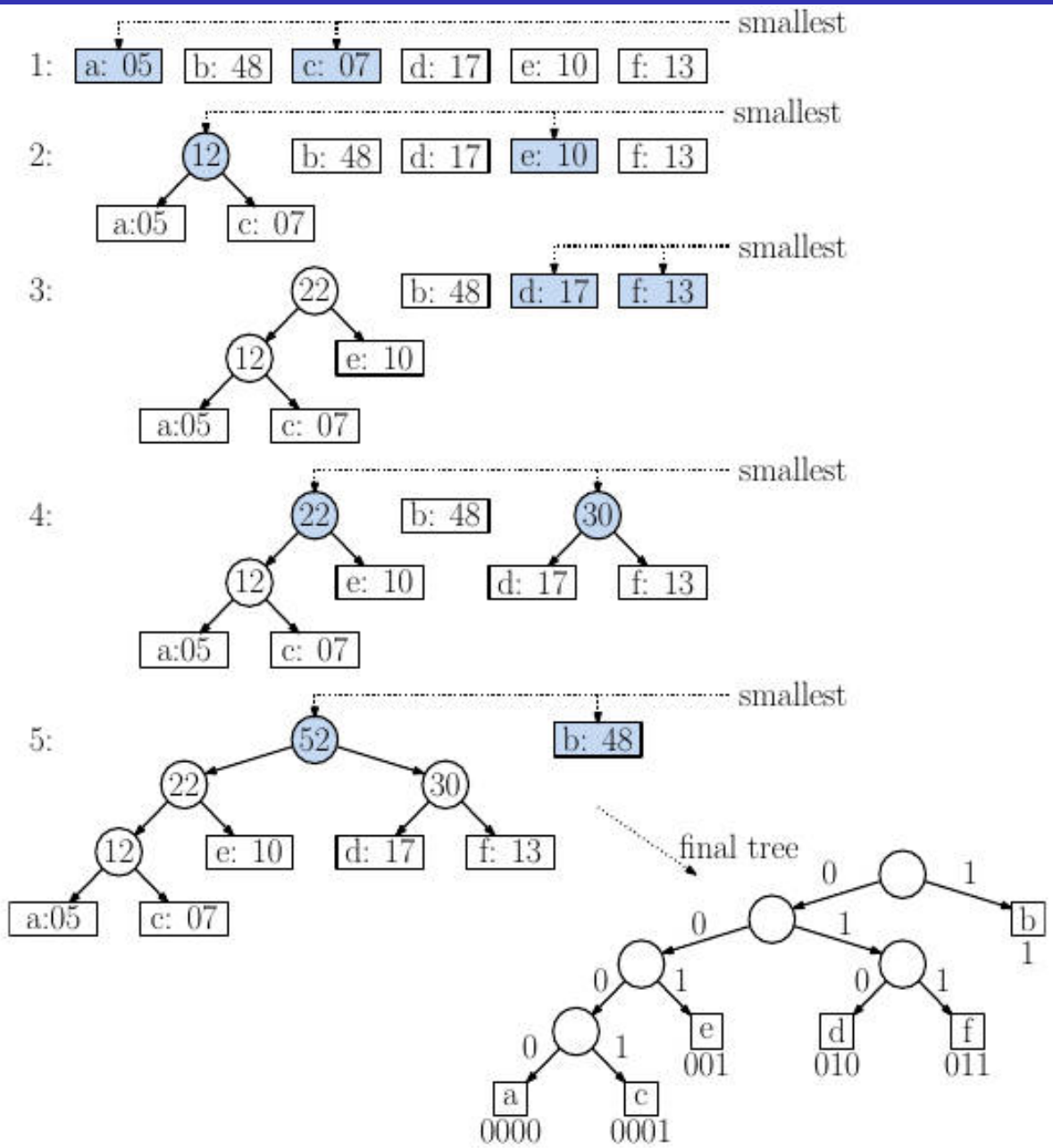
Huffman Encoding Algorithm

How to build an optimal Prefix Tree? Use induction

- Observe that, low probability characters can have long length, thus, put two such lowest probability characters x and y at the leaves of the tree and create a super node z as their parent having sum of probabilities of x nd y.
- Add z into list of remaining characters in C after removing x and y from C.
- Repeat the above steps untill all characters have been considered



Huffman Encoding Algorithm Example



Huffman Encoding Algorithm Pseudocode

```
huffman(char C[], float prob[]) {
    for each (x in C) {
        add x to Q sorted by prob[x]      // add all to priority queue
    }
    n = size of C
    for (i = 1 to n-1) {                  // repeat until 1 item in queue
        z = new internal tree node
        left[z]  = x = extract-min from Q // extract min probabilities
        right[z] = y = extract-min from Q
        prob[z]  = prob[x] + prob[y]      // z's probability is their sum
        insert z into Q                   // z replaces x and y
    }
    return the last element left in Q as the root
}
```

Data structure: Need to quickly obtain two smallest probability characters, remove them from list and add a super character in the list for further iteration

- Priority queue: Min Heap on probability values. Build Heap(), Insert() and Extract-Min() takes $O(n), O(\log n), O(\log n)$ time
- Time complexity of Huffman coding algorithm: $O(n \log n)$

Huffman Encoding Algorithm: Proof of correctness (1/5)

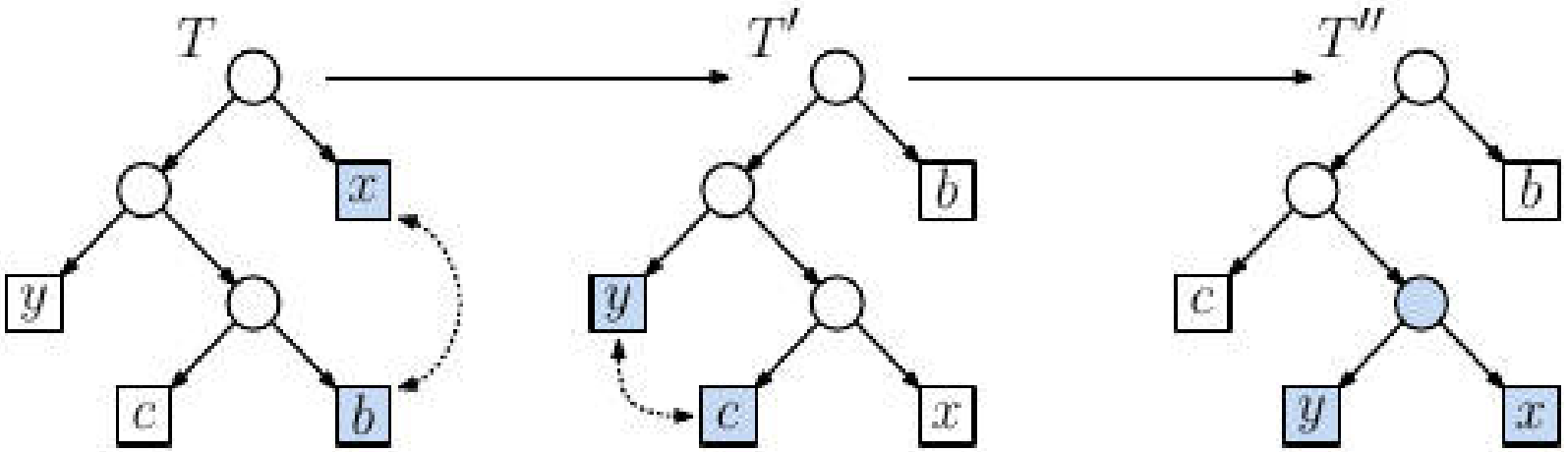
- To show that any tree that differs from the one constructed by Huffman's algorithm can be converted into one that is equal to Huffman's tree without increasing its cost $B(T)$
- Can not use the trick of replacing the optimal partial structure of solution by greedy choice as in Scheduling problem
- Use an induction proof
- **Observation 1:** Huffman tree is a full binary tree, i.e, every internal node has exactly two children.
- **Observation 2:** In any optimal code tree, the two characters with the lowest probabilities will be siblings at the maximum depth in the tree. As per algorithm these two characters are merged into a single super character whose probability is the sum of their individual probabilities. i.e, there is one less character in the alphabet. Thus apply induction to the remaining $n - 1$ characters.

Huffman Encoding Algorithm: Proof of correctness (2/5)

Claim 1: Let x and y are two characters with the smallest probabilities. Then there is an optimal code tree in which these two characters are siblings at the maximum depth in the tree.

- Let T be any optimal prefix code tree, and let b and c be two siblings at the **maximum depth** of the tree.
- Assumption: without loss of generality that $p(b) \leq p(c)$ and $p(x) \leq p(y)$; if not swap them.
- Since x and y have the two smallest probabilities it follows that $p(x) \leq p(b)$ and $p(y) \leq p(c)$. $=$
- Because b and c are at the deepest level of the tree, $d_T(b) \geq d_T(x)$ and $d_T(c) \geq d_T(y)$.
- Thus, $p(b) - p(x) \geq 0$ and $d_T(b) - d_T(x) \geq 0$, and hence their product is nonnegative.
- Now, switch the positions of x and b in the tree, resulting tree T'
- Cost of T', $B(T')$ is the $B(T)-$ cost of x and b in T + cost of x and b in T', as rest of the characters have not changed their positions.
- $B(T') = B(T) - p(x)d_T(x) + p(x)d_T(b) - p(b)d_T(b) + p(b)d_T(x)$

Huffman Encoding Algorithm: Proof of correctness (3/5)



$$B(T') = B(T) - p(x)d_T(x) + p(x)d_T(b) - p(b)d_T(b) + p(b)d_T(x)$$
$$= B(T) + p(x)(d_T(b) - d_T(x)) - p(b)(d_T(b) - d_T(x))$$
$$= B(T) - (p(b) - p(x))(d_T(b) - d_T(x))$$

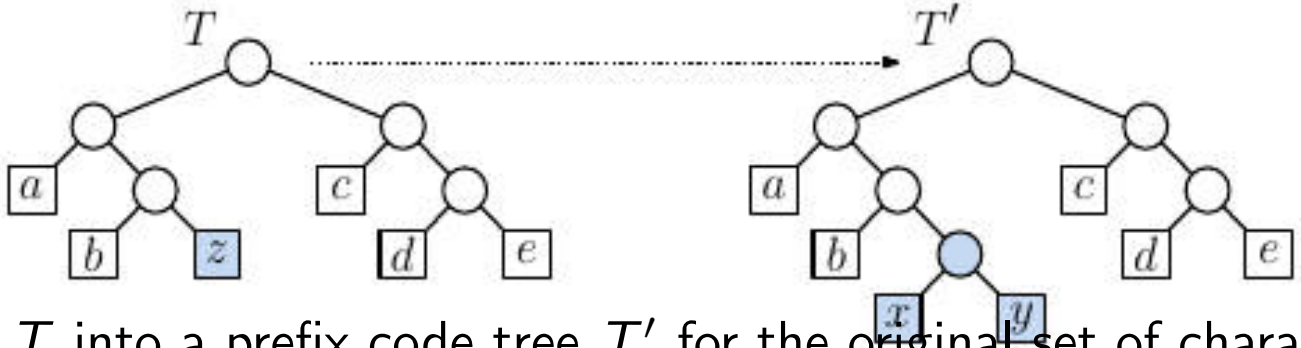
- So, $B(T') \leq B(T)$, (because $(p(b) - p(x))(d_T(b) - d_T(x)) \geq 0$)
- Since T was an optimal tree from our initial assumption, T' is also an optimal tree (cost of T cannot decrease).
- Similarly, switch y with c to obtain a new tree T'', which is also optimal. The final tree T'' satisfies the statement of the claim.

Huffman Encoding Algorithm: Proof of correctness (4/5)

Claim 2: Huffman's algorithm produces an optimal prefix code tree.
Proof by induction on number of characters n

- **Recursive view:** Merge the two lowest probability characters x and y into a single "super character" z whose probability is the sum of their individual probabilities, and then recursively apply the same algorithm to the $n-1$ element character set with x and y replaced by z
- **Basis case:** $n = 1$, the tree consists of a **single leaf node**, which is optimal.
- **Induction hypothesis:** given $n-1$ characters, Huffman's algorithm produces an optimal tree.
- Show that hypothesis is true when the alphabet has exactly n characters.
- When we have n characters, let x and y be the two characters with the lowest probabilities and by Claim 1, they are siblings at the lowest level of the optimal tree.
- As per algo, x and y is removed from the alphabet, and replaced by z whose probability is $p(z) = p(x) + p(y)$; thus, $n-1$ characters in alphabet.
- Consider any prefix code tree T made with this new set of $n-1$ characters, where z will appear as a leaf node somewhere in this tree.

Huffman Encoding Algorithm: Proof of correctness (5/5)



- Convert T into a prefix code tree T' for the original set of characters by replacing the leaf node for z with an internal node whose left child is x and whose right child is y. Let $D_T(z)$ is depth of z
- Cost of new tree T' is

$$B(T') = B(T) - p(z)d_T(z) + p(x)(d_T(z)+1) + p(y)(d_T(z)+1)$$

$$= B(T) - (p(x)+p(y))d_T(z) + (p(x)+p(y))(d_T(z)+1)$$

$$= B(T) + (p(x)+p(y))(d_T(z)+1-d_(z))$$

$$= B(T) + (p(x)+p(y))$$

- The change in cost does not depend on the structure of the tree T. Therefore, in order to minimize $B(T')$, build T to minimize $B(T)$.
- By induction hypothesis and since T involves $n-1$ characters, T results from running Huffman's algorithm is optimal.

Table of Contents

1. Greedy Algorithm

2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree

3. References

Table of Contents

1. Greedy Algorithm
2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree
3. References

Optimal Matrix Chain Order

Problem statement: Given a sequence (chain) $< A_1, A_2, \cdots A_n >$ of n matrices to be multiplied, where the matrices aren't necessarily square, the goal is to compute the product $A_1.A_2.\cdots A_n >$ **minimizing the number of scalar multiplications**.

Note: Multiplications of two matrices will be done in usual way with $\Theta(n^3)$ time complexity, we need to decide in which we would multiply two matrices such that **number of multiplications is minimum**

- Given the matrices A_1, A_2, A_3, A_4, the number of ways they can be **ordered** for multiplications are:

$$(A_1(A_2(A_3A_4)))$$

$$(A_1((A_2A_3)A_4))$$

$$((A_1A_2)(A_3A_4))$$

$$((A_1(A_2A_3))A_4)$$

$$(((A_1A_2)A_3)A_4)$$

- The problem is to compute the number of ways of **fully parenthesizing** the sequence of A_i

Optimal Matrix Chain Order: Problem

- **Note:** the number of multiplications required to multiply $A_{p\times q}$ and $B_{q\times r}$ is pqr.
- Let the matrices A_1, A_2, A_3 have dimensions $10 \times 100, 100 \times 5, 5 \times 50$ respectively, then the number of ways they can be **ordered** for multiplications are: a)$((A_1A_2)A_3)$,b)$(A_1(A_2A_3))$
- The number of multiplications required are:
  $10 \times 100 \times 5 + 10 \times 5 \times 50 = 5000 + 2500$ and
  $100 \times 5 \times 50 + 10 \times 100 \times 50 = 25000 + 50000$ respectively; **Thus a)**$((A_1A_2)A_3)$ **requires minimum number of multiplications (**7500**)**;

Problem statement re-stated: Given a chain $< A_1, A_2, \cdots A_n >$ of n matrices, $i = 1, 2, \cdots n$, where matrix A_i has dimension $p_{i-1} \times p_i$, **fully parenthesize** the product $A_1.A_2.\cdots.A_n$ in a way that minimizes the number of scalar multiplications. The input is the sequence of dimensions $< p_0, p_1, p_2, \cdots p_n >$.

Counting number of possible full parenthesization

- Think of a recursion to count parenthesization: Let $P(n)$ be the **number of alternative parenthesizations of a sequence of n matrices**.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.(**base condition**)
- For $n \geq 2$ a fully parenthesized matrix product is the **product of two fully parenthesized matrix subproducts,** and the split between the two subproducts may occur between the kth and $k + 1^{st}$ matrices for any $k = 1, 2, \cdots n - 1$

$$P(n) = \sum_{k=1}^{n-1} P(k)P(n-k), \text{for} n \geq 2$$
$$= 1, \text{for} n = 1$$

- Above trurns out to be $(n-1)^{th}$ **Catalan Number** $C(n-1) = (2(n-1))!/(n!(n-1)!) = \Omega(4^n/n^{3/2})$, exponential!

Properties of Dynamic Programming

- **Optimal substructure:** a problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems. Optimal substructure varies across problem domains in two ways:
  - how many subproblems an optimal solution to the original problem uses, and
  - how many choices you have in determining which subproblem(s) to use in an optimal solution
- **Overlapping subproblems:** When a recursive algorithm revisits the same problem repeatedly, we say that the optimization problem has overlapping subproblems. (in divide and conquer it recurse on distinct partitions of data)

Applying Dynamic Programming

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Bottom-up implementation to compute the value of an optimal solution.
4. Construct/extract the optimal solution from computed information.

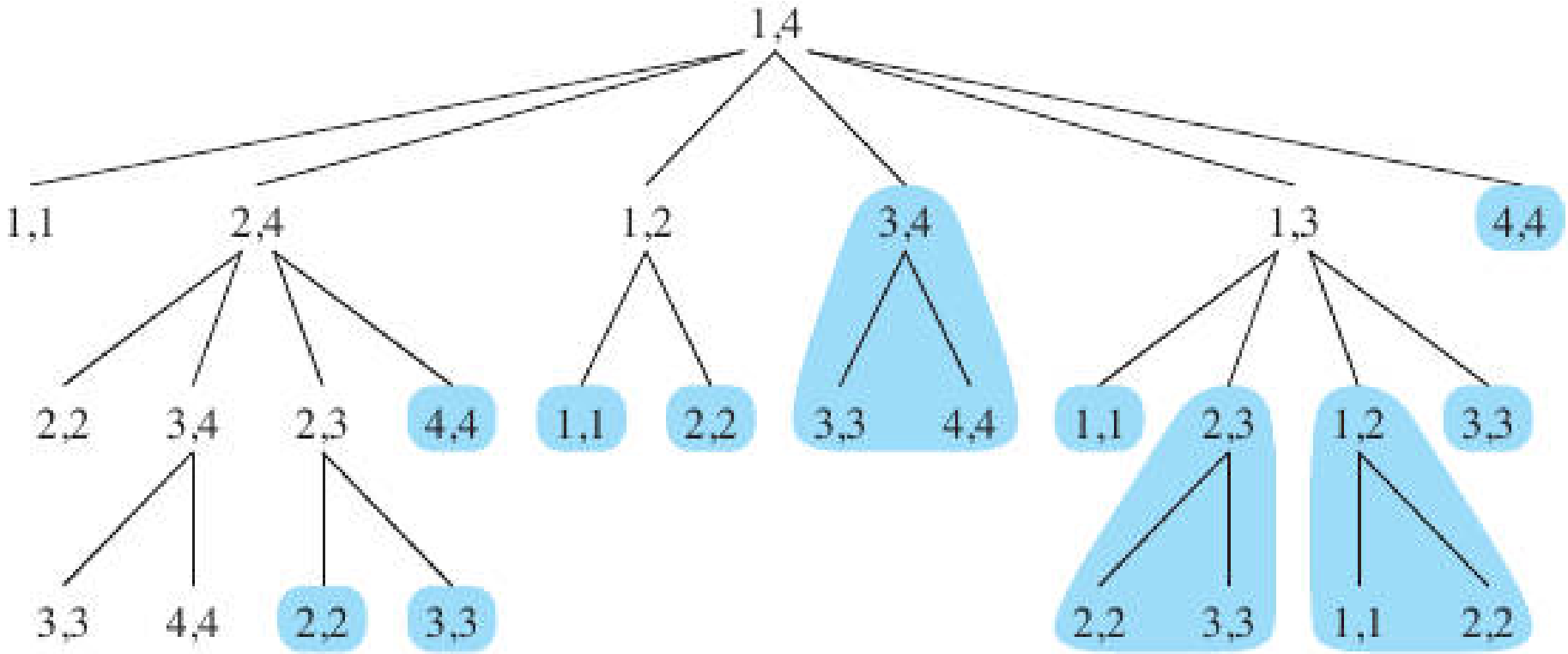
Dynamic Programming for Matrix chain order: Step 1

Step 1: The structure of an optimal parenthesization

- Let $A_{i:j}$ denotes the matrix that results from evaluating the product $A_i.A_i + 1.\cdots.A_j$, for $i \leq j$. Note that result of $A_{i:j}$ is a $p_{i-1} \times p_j$ matrix.
- Let k, where $i \leq k < j$ is a point of split between $A_{i:k}$ and $A_{k+1:j}$. Thus, $A_{1:n} = A_{i:k}.A_{k+1:n}$
- Find **optimal paranthesization**for $A_{i:k}$ and $A_{k+1} : n$, then combine the solutions to get the optimal parenthesization for $A_{i:n}$.
- **Question:** What is the best place to split the chain? (what is k?)
- **Question:** How do we parenthesize each of the subsequences $A_{1:k}$ and $A_{k+1:n}$?

Optimal substructure and overlapping subproblem

Matrix chain : $< A_1 A_2 A_3 A_4 >$



Dynamic Programming for Matrix chain order: Step 2

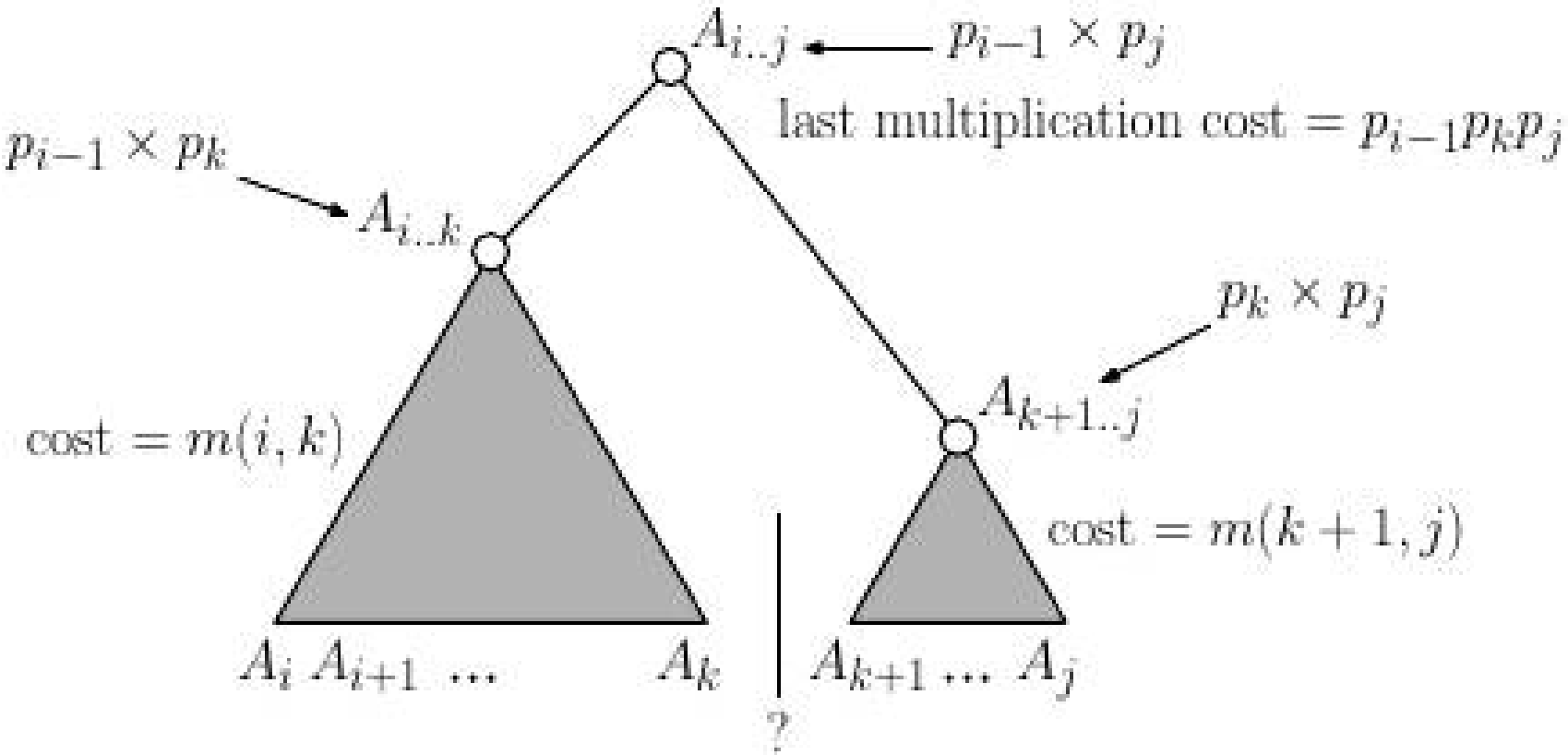
Step 2: Recursively define value of optimal parenthesization

- Let $m(i, j)$ denote the minimum number of multiplications needed to compute $A_{i:j}$, $1 \leq i \leq j \leq n$.
- For entire chain $A_{1:n}$, the minimum number of multiplication (cost) will be given by $m(1, n)$
- Minimum cost is defined as a recursive function:
  - **Basis step:** if $i = j$, there is one matrix and thus, $m(i, i) = 0$
  - **Otherwise:** If $i < j$, $A_{i:j}$ can be split into two groups $A_{i:k} \times A_{k+1:j}$ by considering each k, $i \leq k < j$
  - **Note:** Minimum cost for $A_{i:k}$ and $A_{k+1:j}$ are $m(i, k)$ and $m(k + 1, j)$ which have been computed previously and are already stored in the array m
  - **Note:** Since $A_{i:k}$ is a $p_{i-1} \times p_k$ matrix, and $A_{k+1:j}$ is a $p_k \times p_j$ matrix, the number of multiplications required to multiply them is $p_{i-1} p_k p_j$.
  - **The total cost for** $A_{i:j} = m(i, k) + m(k + 1, j) + p_{i-1} p_k p_j$

Dynamic Programming for Matrix chain order: Step 2

Step 2: Recursively define value of optimal parenthesization

$$
\begin{aligned}
m(i,i) &= 0 \\
m(i,j) &= \min_{i \le k < j} \left(m(i,k) + m(k+1,j) + p_{i-1} p_k p_j \right) \qquad \text{for } i < j.
\end{aligned}
$$



Dynamic Programming for Matrix chain order: Step 3

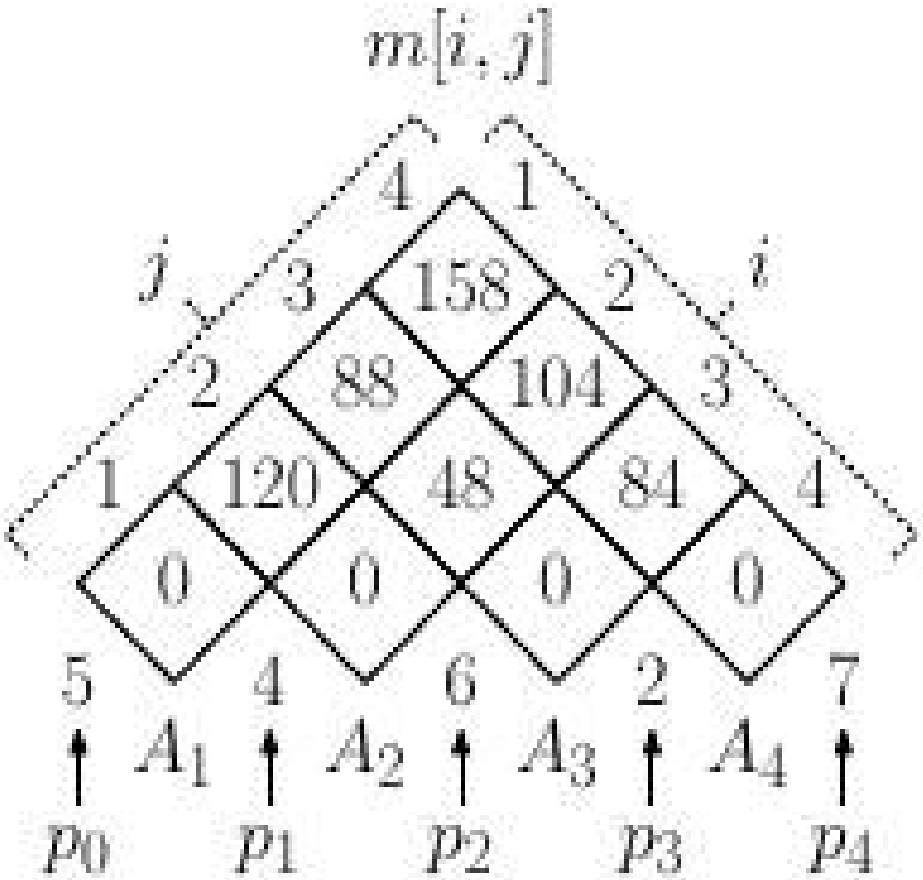
Step 3: Bottom up iterative implementation: In which order to fill up $m(i,j)$? Matrix sequence of length 2, length 3, till n. **Note** a length 2 optimal subproblem is a subproblem of length 3 subproblem, that is solved first to find an optimal solution for length 3 subproblem

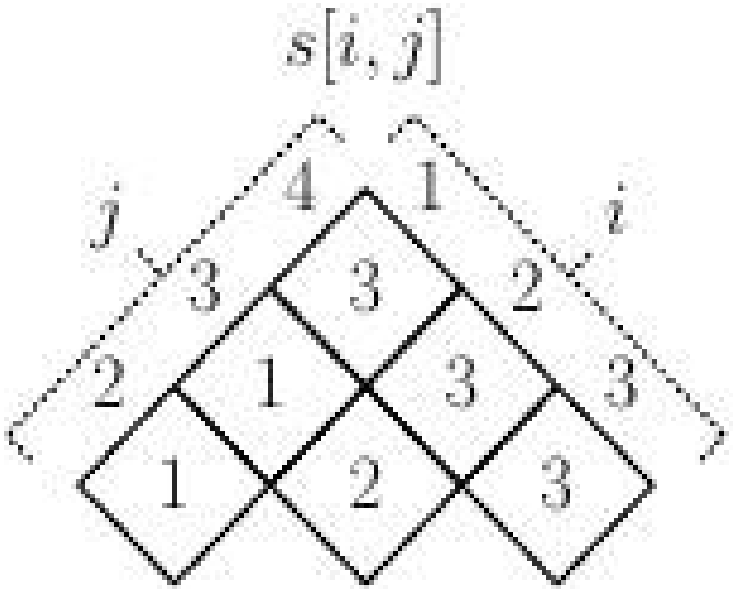
```
Matrix-Chain(p[0..n]) {
    s = array[1..n-1, 2..n]
    for (i = 1 to n) m[i, i] = 0                    // initialize
    for (L = 2 to n) {                              // L = length of subchain
        for (i = 1 to n - L + 1) {
            j = i + L - 1
            m[i,j] = INFINITY
            for (k = i to j - 1) {                  // check all splits
                cost = m[i, k] + m[k+1, j] + p[i-1]*p[k]*p[j]
                if (cost < m[i, j]) {               // found a new optimum?
                    m[i, j] = cost                  // ...save its cost
                    s[i, j] = k                     // ...and the split marker
                }
            }
        }
    }
    return m[1, n] (final cost) and s (splitting markers)
}
```

Dynamic Programming for Matrix chain order: Step 3

Time complexity: $O(n^3)$

Example:



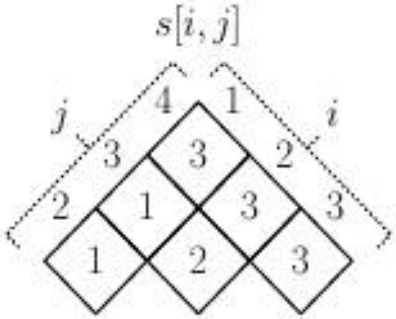


Dynamic Programming for Matrix chain order: Step 4

Step 4: Construct/Extract the optimal parenthesization

- $s(i,j), j > i$ stores the value of k that provides the optimal split, best way to multiply the two sequences $A_{i:k}$ and $A_{k+1:j}$; Intuitively, $s(i,j)$ tells what multiplication to perform **last**.
- Time complexity: $O(n)$

```
do-mult(i, j) {
    if (i == j)                          // basis case
        return A[i]
    else {
        k = s[i,j]
        X = do-mult(i, k)                // X = A[i]...A[k]
        Y = do-mult(k+1, j)              // Y = A[k+1]...A[j]
        return X * Y                     // multiply matrices X and Y
    }
}
```



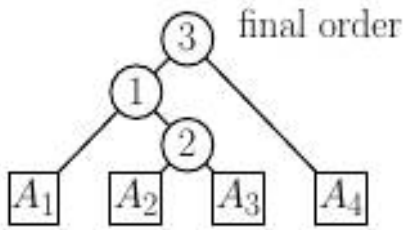


Table of Contents

1. Greedy Algorithm
2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree
3. References

All Pair Shortest Path

- Generalization of the shortest path problem: compute shortest paths between all pairs of vertices
- Let $G = (V, E)$ be a **weighted directed graph**;If $(u, v) \in E$, is an edge of G, then the weight of this edge is denoted $w(u, v)$.
- The **cost of a path** is the **sum of edge weights along the path**; the distance between two vertices $\delta(u, v)$ is the **cost of the minimum cost path between them**.
- G may have **negative cost edges**, but **no negative cost cycles**.
- **Problem:** Determine the cost of the shortest path between all pairs of vertices in G.
- Since dynamic programming wil be used, **adjacency matrix** representation of G will be more useful.

All Pair Shortest Path: Input and Output

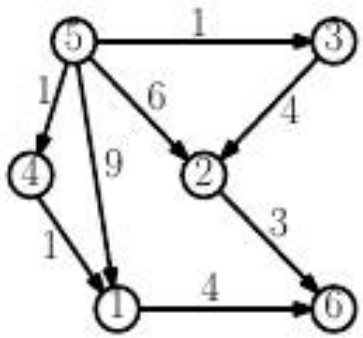
- Input is an $n \times n$ matrix w of edge weights

$$w(i,j) = \begin{cases} 0 & \text{if} i = j, \\ w(i,j) & \text{if } i \neq j \text{ and } (i,j) \in E, \\ +\infty & \text{if } i \neq j \text{ and } (i,j) \notin E, \end{cases} \quad (1)$$

- Output is an $n \times n$ distance matrix $D = d_{ij}$ where $d_{ij} = \delta(i,j)$, the shortest path cost from vertex i to j.
- To extract the shortest path compute an **auxiliary matrix** $mid[i,j]$ that stores a vertex that is somewhere along the shortest path from i to j.
- If the shortest path travels directly from i to j without passing through any other vertices, then $mid[i,j] = Null$

Floyd-Warshall Algorithm : DP Formulation

- The Floyd-Warshall algorithm runs in $\Theta(n^3)$ time.
- Find the best formulation for the shortest path subproblem by limiting the set of vertices through which the path is allowed to pass. For a path $p =< v_1, v_2, \cdots, v_i >$,vertices $< v_1, v_2, \cdots, v_{i-1} >$ are the **intermediate vertices** of this path.
- **DP formulation:** Define $d_{ij}^{(k)}$ to be the **shortest path from** i **to** j such that any intermediate vertices on the path are **chosen from the set** $\{1, 2, \cdots, k\}$.
- Limiting intermediate vertices: $d_{5,6}^{(3)}$ can go through any combination of intermediate vertices $\{1, 2, 3\}$, of which $< 5, 3, 2, 6 >$ has lowest cost of 8.



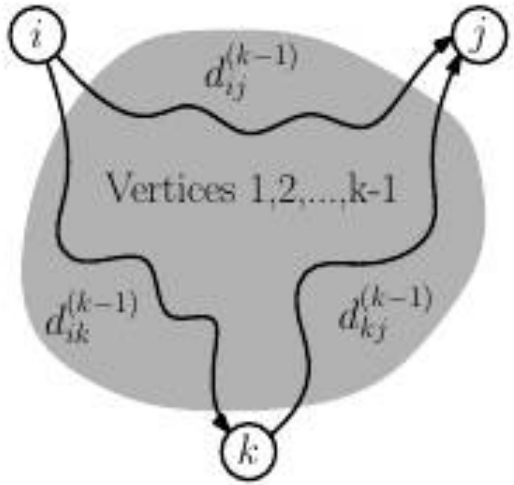
$d_{5,6}^{(0)} = \infty$ (no path)

$d_{5,6}^{(1)} = 13$ $\langle 5, 1, 6 \rangle$

$d_{5,6}^{(2)} = 9$ $\langle 5, 2, 6 \rangle$

$d_{5,6}^{(3)} = 8$ $\langle 5, 3, 2, 6 \rangle$

$d_{5,6}^{(4)} = 6$ $\langle 5, 4, 1, 6 \rangle$



DP: How to compute $d_{ij}^{(k)}$?

Given $d_{ij}^{(k-1)}$, compute $d_{ij}^{(k)}$ based on two choices:

- **Don't go through k at all:** The shortest path from node i to node j uses intermediate $(k-1)$ vertices $\{1, \cdots, k-1\}$, and hence the length of shortest path is $d_{ij}^{(k-1)}$.
- **Do go through k:** We pass through k exactly once for shortest path (since no negative cost cycles). For overall shortest path take the shortest path from i to k, and the shortest path from k to j. Since these paths uses $(k-1)$ intermediate vertices only in $\{1, 2, \cdots, k-1\}$, the length of the path is $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

DP Recurrence:

$$\begin{aligned} d_{ij}^{(0)} &= w_{ij}, \\ d_{ij}^{(k)} &= \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) \qquad \text{for } k \geq 1. \end{aligned}$$

Final Result: $d_{ij}^{(n)}$ because it considers all possible vertices as intermediate vertices.

Floyd-Warshall Algorithm (1)

Bottom-up iterative algorithm instead of recursion using Table lookup, inorder to avoid recomputation of smaller subproblems.

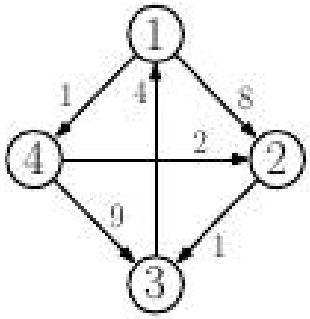
```
Floyd_Warshall(n, w) {
    array d[1..n, 1..n]                          // distance matrix
    for (i = 1 to n) {                           // initialize
        for (j = 1 to n) {
            d[i,j] = W[i,j]
            mid[i,j] = null
        }
    }
    for (k = 1 to n) {                           // use intermediates {1..k}
        for (i = 1 to n) {                       // ...from i
            for (j = 1 to n) {                   // ...to j
                if (d[i,k] + d[k,j]) < d[i,j]) {
                    d[i,j] = d[i,k] + d[k,j]// new shorter path length
                    mid[i,j] = k                 // new path is through k
                }
            }
        }
    }
    return d                                     // final array of distances
}
```

Floyd-Warshall Algorithm (2)

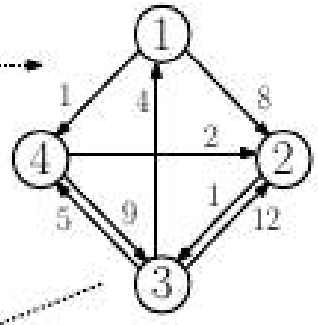
- $mid\,[i,j]$ for extracting the final shortest paths as in other DP formulation. $mid\,[i,j] = k$, whenever **shortest path** goes through vertex k, else $mid\,[i,j] = null$.
- Time Complexity : $\Theta(n^3)$, Space Complexity: $\Theta(n^2)$
- Note that superscript (k) is not used in code. Values are overwritten with same value. The entries in kth row and kth column are overwritten and reused.
- **Extracting shortest path:** To construct the shortest path, if $mid\,[i,j] = null$, shortest path is the edge (i,j), else recursively compute the shortest path from i to $mid\,[i,j]$ and the shortest path from $mid\,[i,j]$ to j. Time complexity is $\Theta(m)$, number of edges.

Floyd-Warshall Algorithm : Example (3)

$H[i,j]$ is same as $mid[i,j]$

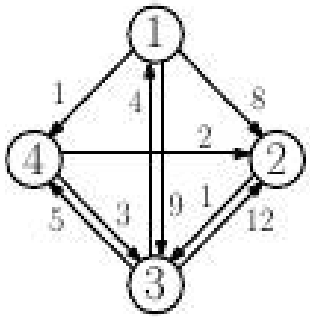


$$d^{(0)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix}$$

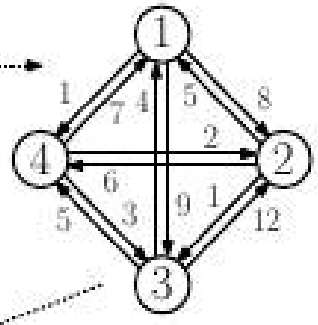


$$d^{(1)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix}$$

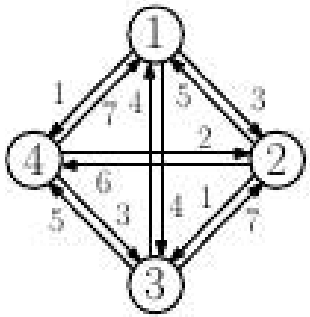
$H[i,j]$



$$d^{(2)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix}$$



$$d^{(3)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$



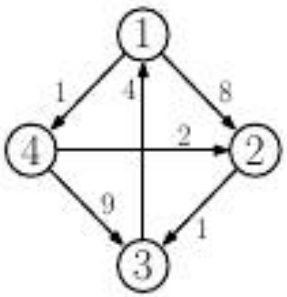
$$d^{(4)} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

$$d = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \qquad H = \begin{bmatrix} \emptyset & 4 & 4 & \emptyset \\ 3 & \emptyset & \emptyset & 3 \\ \emptyset & 4 & \emptyset & 1 \\ 3 & \emptyset & 2 & \emptyset \end{bmatrix}$$

Floyd-Warshall Algorithm : Extract Path (4)

$H[i,j]$ is same as $mid[i,j]$

```
get-path(i, j) {                          // return edges on the shortest path
    if (h[i, j] == null)                  // no intermediate?
        return (i, j)                     // shortest path is a single edge
    else {                                // path goes through h[i,j]
        mid = h[i, j]                     // concatenate paths (i --> mid) + (mid --> j)
        return get-path(i, mid) + get-path(mid, j)
    }
}
```



$$H = \begin{bmatrix} \emptyset & 4 & 4 & \emptyset \\ 3 & \emptyset & \emptyset & 3 \\ \emptyset & 4 & \emptyset & 1 \\ 3 & \emptyset & 2 & \emptyset \end{bmatrix}$$

$\texttt{get-path}(2,4)$: $\texttt{get-path}(2,3) \oplus \texttt{get-path}(3,4)$

$(2,3) \oplus (\texttt{get-path}(3,1) \oplus \texttt{get-path}(1,4))$

$(2,3) \oplus (3,1) \oplus (1,4)$

Fig. 4: Extracting the shortest path from 2 to 4.

Table of Contents

1. Greedy Algorithm
2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree
3. References

Optimal binary search tree: The problem (1)

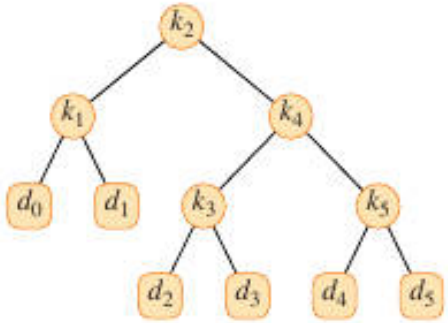
- A **Binary Search Tree (BST)** is constructed to store English words for English to Bengali translation very fast.
- Certain words occur more frequently in the text whose Bengali translation needs to be search very fast; Those words should not be kept as leaf but must be kept near the root for faster search.
- Moreover, some words in the text might have no Bengali translation, and such words would not appear in the binary search tree at all.
- **Problem:** How can you organize a binary search tree so as to **minimize the number of nodes visited in all searches**, given that you know how often each word occurs?
- **Optimal Binary Search Tree Generation Problem:** Given a sequence $K =< k_1, k_2, \cdots, k_n >$ of n distinct keys such that $k_1 < k_2 < \cdots < k_n$, build an optimal binary search tree containing them such that expected search cost is minimum.

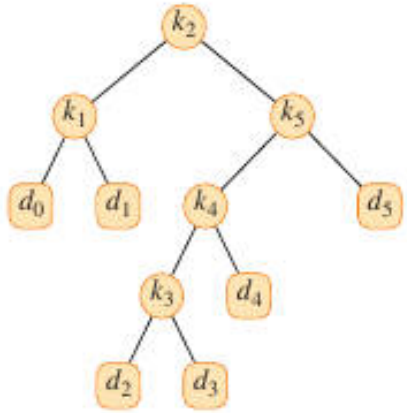
Optimal binary search tree: The problem (2)

- Each key k_i has a **given search probability** p_i.
- For values not in K, there are $n+1$ **dummy keys** $d_0, d_1, \cdots, d_n$. d_0 represents all values $< k_1$, d_n represents all values $> k_n$, for $i = 1, 2, \cdots n-1$, $k_i < d_i < k_{i+1}$. For each dummy key d_i, the **search probability is** q_i.
- Each key k_i is an **internal node**, and each dummy key d_i is a **leaf**. Since every search is either successful (for some key k_i) or unsuccessful (some dummy key d_i), we have $\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1$
- **Expected Cost of a search in a given BST:** Let **actual cost of a search** equals the number of nodes examined, (the depth of the node found by the search in T) $+1$.
- E [search cost in T] $= \sum_{i=1}^{n} (depth_T(k_i) + 1).p_i + \sum_{i=0}^{n} (depth_T(d_i) + 1).q_i$

  E [search cost in T] $= 1 + \sum_{i=1}^{n} depth_T(k_i).p_i + \sum_{i=0}^{n} depth_T(d_i).q_i$
- **Optimal binary search tree:** For a given set of probabilities, construct a binary search tree with smallest expected search cost.

Example: Optimal BST





node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

(a)

node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	3	0.05	0.20
k_4	2	0.10	0.30
k_5	1	0.20	0.40
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	4	0.05	0.25
d_3	4	0.05	0.25
d_4	3	0.05	0.20
d_5	2	0.10	0.30
Total			2.75

(b)

Figure 14.9 Two binary search trees for a set of $n = 5$ keys with the following probabilities:

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

(a) A binary search tree with expected search cost 2.80. **(b)** A binary search tree with expected search cost 2.75. This tree is optimal.

DP Solution for Optimal binary search tree problem

Step 1: Structure of optimal BST

- Consider any subtree of a binary search tree that contain keys in a contiguous range $k_i, \cdots, k_j$, for some $1 \leq i \leq j \leq n$. It must also have as its leaves the dummy keys $d_{i-1}, \cdots, d_j$.
- **optimal substructure:** if an optimal BST T has a subtree T' containing keys $k_i, \cdots, k_j$, then T' must be optimal for the subproblem with keys $k_i, \cdots, k_j$ and dummy keys $d_{i-1}, \cdots, d_j$.
- Let k_r is the key at the root of the optimal subtree T'. For each possible $i \leq r \leq j$, create optimal subtree with keys, $k_i, \cdots, k_{r-1}$ and dummies $d_{i-1}, \cdots, d_{r-1}$ as left subtree of k_r and optimal subtree with keys, $k_{r+1}, \cdots, k_j$ and dummies $d_r, \cdots, d_j$ as right subtree to get the optimal subtree T'

DP Solution for Optimal binary search tree problem

Step 2: Recurrence relation

- **Expected cost** $e[i,j]$ **of optimal subtree** T storing keys $k_i, \cdots, k_j$, and $d_{i-1}, \cdots, d_j$. Compute $e[1,n]$ to get the final optimal BST.
- When $j = i-1$ for $k_i, \cdots, k_j$, the subproblem consists of just the dummy key d_{i-1}. The expected search cost is $e[i, i-1] = q_{i-1}$
- Selct each k_r from $k_i, \cdots k_j$ as the root, find optimal BST at left and right subtree of k_r with appropriate keys and dummies; the depth of each node in subtrees increases by 1 as it is added to the root node.
- **The expected search cost of the subtrees increases by**

  $$w(i,j) = \sum_{l=i}^{j} p_l + \sum_{l=i-1}^{j} q_l$$

- $e[i,j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j))$

  $e[i,j] = e[i, r-1] + w(i,j) + e[r+1, j]$

$$e[i,j] = \begin{cases} q_{i-1} & \text{if } j = i-1, \\ \min\{e[i,r-1] + e[r+1,j] + w(i,j) : i \le r \le j\} & \text{if } i \le j. \end{cases}$$

DP Solution for Optimal binary search tree problem

Step 3: Computing expected search cost: Like matrix-chain order- length 1 , length 2 and so on until n for $k_1, \cdots k_n$.

```
OPTIMAL-BST(p, q, n)
 1  let e[1 : n + 1, 0 : n], w[1 : n + 1, 0 : n],
        and root[1 : n, 1 : n] be new tables
 2  for i = 1 to n + 1                    // base cases
 3      e[i, i − 1] = q_{i−1}             // equation (14.14)
 4      w[i, i − 1] = q_{i−1}
 5  for l = 1 to n
 6      for i = 1 to n − l + 1
 7          j = i + l − 1
 8          e[i, j] = ∞
 9          w[i, j] = w[i, j − 1] + p_j + q_j        // equation (14.15)
10          for r = i to j                           // try all possible roots r
11              t = e[i, r − 1] + e[r + 1, j] + w[i, j]   // equation (14.14)
12              if t < e[i, j]                       // new minimum?
13                  e[i, j] = t
14                  root[i, j] = r
15  return e and root
```

Time complexity: $\Theta(n^3)$

Step 4: Generate the optimal BST by looking at $root(i, j)$, that stores the selected k_r for root of optimal subtrees.

Table of Contents

1. Greedy Algorithm

2. Dynamic Programming
   - Optimal matrix chain order for minimum multiplications
   - All Pair Shortest path
   - Optimal binary Search Tree

3. References

References

- Lecture notes on Design and Analysis of Algorithm , David Mount, 2015, Lecture 7(Greedy Huffman Algorithm), Lecture 15 (DP: Matrix chain order), Lecture 16 (DP: All pair shortest path)
- Introduction to Algorithms, T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein,Prentice Hall India, Fourth Edition, Chapter 15.2, 15.2 (Greedy Huffman Algorithm), Chapter 14.2, 14.3 (DP: Matrix chain order), Chapter 23.2 (DP: All pair shortest path), Chapter 14.5 (DP: Optimal BST)